

Midterm Report

Yiyou Liu

Mechanical Engineering (Robotics)
Guangdong Technion-IIT
Shantou, China
liu15183@gtiit.edu.cn

Yimeng Zhang

Mechanical Engineering (Robotics)
Guangdong Technion-IIT
Shantou, China
yimeng.zhang@gtiit.edu.cn

Wuhao Cai

Mechanical Engineering (Robotics)
Guangdong Technion-IIT
Shantou, China
wuhao.cai@gtiit.edu.cn

Deqing RUAN

Mechanical Engineering (Robotics)
Guangdong Technion-IIT
Shantou, China
ruan14954@gtiit.edu.cn

Gening Liu

Mechanical Engineering (Robotics)
Guangdong Technion-IIT
Shantou, China
liu15555@gtiit.edu.cn

Abstract: Bipedal locomotion on discontinuous terrain is a challenging task due to the requirements for stable control and accurate foothold placement. This paper presents a deep reinforcement learning approach for bipedal walking on stepping-stone terrain using depth-based perception. A depth camera is used to directly observe the terrain in an end-to-end manner, without relying on explicit environment mapping or SLAM. The visual input is combined with the robot's internal state and fed into a learned control policy.

The locomotion policy is trained using Proximal Policy Optimization within a two-stage training framework. In the first stage, the robot learns a stable blind walking gait on flat ground to acquire fundamental locomotion skills. In the second stage, depth perception is enabled and a residual learning strategy is adopted to adapt the pre-trained policy to discontinuous stepping-stone terrain. This staged training design improves learning stability and prevents the loss of basic walking capability.

Simulation and real-world experiments demonstrate that the proposed method enables stable flat-ground walking and successful traversal of stepping-stone terrain, validating the effectiveness of the reinforcement learning framework and the two-stage training strategy.

I. INTRODUCTION

This work investigates bipedal locomotion on discontinuous stepping-stone terrain using depth-based perception and deep reinforcement learning. A depth camera is used to directly observe the terrain in an end-to-end manner, without explicit mapping or SLAM, and the perception is integrated with the robot's internal state for control.

The policy is trained using the Proximal Policy Optimization algorithm under a two-stage training framework. In Stage 1, the robot learns a long-stride blind walking gait on flat ground to establish stable forward locomotion. In Stage 2, the depth camera is enabled and the policy is trained to traverse stepping-

stone terrain, where visual perception is required for safe foothold placement.

This staged approach improves training stability and enables the robot to maintain basic walking ability while adapting to discontinuous terrain.

II. METHODS

A. Reinforcement Learning

A Markov decision process (MDP) is a tuple $M = (S, A, T, r, \gamma, t)$, with state space S (a set that contains all the possible state), action space A (a set that contains all the possible action), transition probabilities $T(s_{k+1}|s_k, a_k)$ describing the dynamics of the system (the probability of going into the next desired state with a given current state and action s_k, a_k ($s_k \in S, a_k \in A$), reward function r and discount factor γ . The reward function r takes the general form $r : S \times A \times S \rightarrow \mathbb{R}$ with $r(s_k, a_k, s_{k+1})$ describing the reward received for transitioning to state s_{k+1} from s_k with action a_k . For a given MDP, we wish to find a policy $\pi_\theta : S \rightarrow A$ to maximize the discounted sum of all future rewards, where θ is the function parameter. For a fixed policy, the sum of the discounted future reward of one sampled trajectory can be represented by G_t .

$$G(t) = \sum_{k=0}^{\infty} \gamma^k r_k \quad (1)$$

The expected future return of a specific state s can be represented as state value function $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau) | s_t = s] \quad (2)$$

RL wants to find an optimal policy π^* which is defined below.

$$\pi^* = \underset{\theta}{\operatorname{argmax}} \underset{s \sim \eta}{\mathbb{E}} [V^\pi(s)] = \underset{\theta}{\operatorname{argmax}} J(\theta) \quad (3)$$

To find the optimal function parameters, one can take the gradient of the $J(\theta)$ resulting in Policy Gradient expression, where α is called the learning rate:

$$\nabla_\theta J(\theta) = \underset{\tau \sim \pi_\theta}{\mathbb{E}} \left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t) G(\tau) \right] \quad (4)$$

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta)|_{\theta=\theta_k} \quad (5)$$

Proximal Policy Optimization (PPO) is a kind of Deep Reinforcement Learning (DRL) algorithm that is widely used nowadays, updating policies by taking the largest step possible to improve performance, while satisfying a special constraint on how close the new and old policies are allowed to be. The constraint is like Kullback–Leibler Divergence, a measure of (something like, but not exactly) distance between probability distributions. PPO uses simple value clip constraint that reaches the same performance.

This is different from the normal policy gradient $\nabla_\theta J(\theta)$, which keeps new and old policies close in parameter space. But even seemingly small differences in parameter space can have very large differences in performance—so a single bad step can collapse the policy performance. This makes it dangerous to use large step sizes with vanilla policy gradients, thus hurting its sample efficiency. PPO nicely avoids this kind of collapse and tends to quickly and monotonically improve performance.

The new target function in PPO is described as

$$\theta_{k+1} = \underset{\theta}{\operatorname{argmax}} \underset{s, a \sim \pi_{\theta_k}}{\mathbb{E}} [L(s, a, \theta_k, \theta)] \quad (6)$$

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, \operatorname{clip} \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \varepsilon, 1 + \varepsilon \right) \right) A^{\pi_{\theta_k}}(s, a) \quad (7)$$

$A^{\pi_{\theta_k}}(s, a)$ is called the Generalized Advantage Estimation (GAE), λ is called the GAE factor. For a GAE factor equals to 1, the sampling process is completely Monte Carlo approach. For a GAE factor equals 0. It is completely a Temporal Difference approach.

$$A^{\pi_{\theta_k}}(s_t, a) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta^V_{t+l} \quad (8)$$

$$\delta^V_t = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t) \quad (9)$$

B. State Space and Action Space

The robot could observe the following state:

TABLE I. ROBOT STATE

State Name	Symbols	Dimension
Joint Angle	θ	8
Joint Angular Velocity	$\dot{\theta}$	8
ZYX Euler Angle	<i>roll, pitch, yaw</i>	3
Body Angular Velocity	ω	3
Sine Clock	$\sin(\phi)$	1
Cosine Clock	$\cos(\phi)$	1
Previous Actions	a_{t-1}	8
Velocity Cmd	v_{cmd}	1
Depth Image	D	11×18

The variable ϕ in $[0, 1)$ represents the normalized phase of the walking gait cycle. It is defined as the elapsed time modulo the gait period, divided by the period. To provide a continuous and unique phase signal to the policy, the sine and cosine of $2\pi\phi$ are computed and provided as observations. This encoding avoids the discontinuity inherent in the raw phase value as it wraps from 1 back to 0.

The action space is defined by a_t , which represents the target joint angle.

A. Low Level Control

The joint torque is computed using PD control in the form of the following:

$$\tau = K_p(a_t - \theta) + K_d(0 - \dot{\theta}) \quad (10)$$

The PD controller is running at 500 Hz, While the RL is running at 50 Hz.

B. Two Stage Training

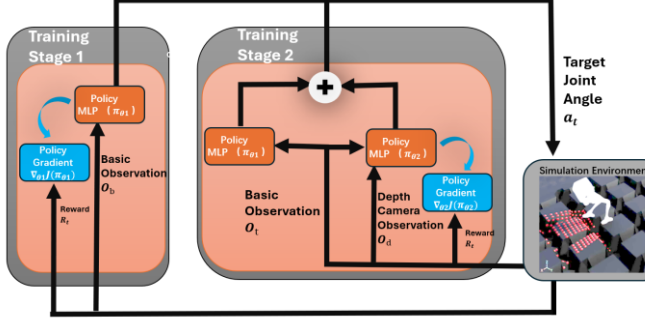


Fig.1 Training Framework

The training procedure is conducted in two distinct stages to facilitate efficient and robust policy learning.

In the first stage, the depth camera is disabled, and the robot is trained exclusively on a flat, uniform terrain. This stage aims to teach robot fundamental locomotion skills, such as balance maintenance and stable walking. The policy learned in this phase is denoted as π_{θ_1} .

In the second stage, the depth camera is enabled, and the training environment is augmented with challenging stepping-stone terrain. To adapt the flat-terrain policy to this complex scenario, we employ a residual learning strategy. Instead of learning a new policy from scratch, we introduce a second policy, π_{θ_2} . The final action carried out by the robot is the sum of the output from both policies

Here, π_{θ_1} serves as a pre-trained base policy that provides proficient locomotion primitives. The role of π_{θ_2} is to act as a *regulator* or *residual adapter*. It teaches to output corrective actions that fine-tune the base policy's behavior in response to the newly introduced visual input and terrain complexity. This architecture allows the robot to efficiently learn to negotiate obstacles, such as navigating the gaps between steppingstones, by modulating its established flat-ground walking strategy rather than relearning locomotion entirely.

C. Reward

The policy is trained using a comprehensive, multi-component reward function r_t , designed to encourage stable, energy-efficient, and velocity-tracking locomotion. The total reward at each time step is the sum of several sub-rewards:

$$r_{total} = r_{body} + r_{locomotion} + r_{penalty} \quad (11)$$

$$r_{body} = r_{linear} + r_{angular} \quad (12)$$

$$r_{linear} = r_{cmd} + r_{vel} + r_{height} \quad (13)$$

$$r_{angular} = r_{body\ ori} + r_{foot\ ori} + r_{angvel} \quad (14)$$

$$r_{locomotion} = r_{joint} + r_{foot\ height} + r_{phase1} + r_{phase2} + r_{air} \quad (15)$$

$$r_{penalty} = r_{action\ rate} + r_{termination} \quad (16)$$

TABLE II.

REWARD FUNCTION

Reward	Symbols
$r_{cmd\ tracking}$	$- v_x - 0.5 \cdot \mathbb{I}(v_{cmd} = 1) $
$r_{vel\ constraint}$	$-0.6(v_y + v_z) + 0.5$
r_{height}	$- h - 0.85 + 0.3$
$r_{body\ ori}$	$-0.3(\ roll, pitch\ _2 + yaw)$
$r_{foot\ ori}$	$-0.3(\theta_{Lfoot} + \theta_{Rfoot})$
$r_{ang\ vel}$	$-0.05\ \omega\ _2 + 0.5$
r_{joint}	$-0.6(\theta_{hipL} - 0.1 + \theta_{hipR} + 0.1)$
$r_{foot\ height}$	$4 \sum_{i \in L,R} z_{foot_i} - 0.16 \cdot \mathbb{I}(\text{foot}_i \text{ in air})$
r_{phase1}	$0.3 \cdot \mathbb{I}(\text{single support}) - 0.3 \cdot \mathbb{I}(\text{else})$
r_{phase2}	$\sum_{i \in L,R} 0.3 \cdot \mathbb{I}(i\text{-th leg phase alignment}) - 0.5$
r_{air}	$\sum_{i \in L,R} 5[0.2((T_{air_i} - 0.5) \cdot \mathbb{I}(i\text{-th leg contact}))]$
$r_{action\ rate}$	$-0.01 \cdot a_{t-1} - a_t $
$r_{termination}$	$-10 \cdot \mathbb{I}(\text{terminate})$

III. RESULT

A. Simulation

a. Large-Scale Parallel Training Environment:

The simulation framework is built upon NVIDIA Isaac Lab, utilizing its massive parallelization capabilities to simulate 4,000 bipedal agents concurrently. This setup allows the collection of 200,000 experience samples per training iteration (4,000 agents $\times$ 50 steps), significantly increasing the diversity of the training data compared to traditional single-agent simulations. The physics engine is configured with a 50Hz control frequency ($dt = 0.02s$), matching the target hardware's deployment constraints.

b. Verification of the Two-Stage Residual Learning:

A key focus of our simulation was validating the **Residual Learning** architecture:

$$a_{total} = 0.25 \cdot a_{base} + 0.75 \cdot a_{residual}$$

Stage 1 (Proprioceptive Foundation):

Preliminary results from the first stage (blind walking) demonstrate that the robot successfully learns to prioritize velocity tracking and torso height maintenance. Initial data shows positive rewards in `vel_tracking` (**0.1437**) and `body_height` (**0.2330**), confirming that the reward function correctly guides the robot to establish a stable standing and walking posture.

Stage 2 (Visual Integration):

By introducing the **11x18 depth map**, the simulation demonstrates the robot's potential to perceive terrain gaps. Even in the early stages of training, the residual policy begins to modulate the base gait, allowing the robot to adjust its step length when approaching discontinuous tiles.

3) Sim-to-Real Robustness and Noise Injection:

To prepare for real-world deployment, we implemented extensive **Domain Randomization** and **Sensor Noise** in the simulation. We injected Gaussian noise into the depth camera observations (**0.2 level**) and joint angular velocities (**0.5 level**). Preliminary tests show that the Actor-Critic networks (approx. **3.4MB per model**) are capable of maintaining balance despite these perturbations. This robustness is crucial for handling the inherent sensor inaccuracies of the real robot's IMU and depth sensors.

4) Qualitative Observations:

Snapshots from the simulation (Fig.2 and Fig.3) illustrate the robot's progress. In Stage 1, the robot maintains a steady forward gait on flat terrain with minimal torso oscillation. In Stage 2, the integration of depth perception allows the robot to "anticipate" terrain edges, showcasing the effectiveness of our residual adaptation strategy over a pure end-to-end learning approach.



Fig.2 Simulation of bipedal locomotion on flat terrain in Isaac Sim.

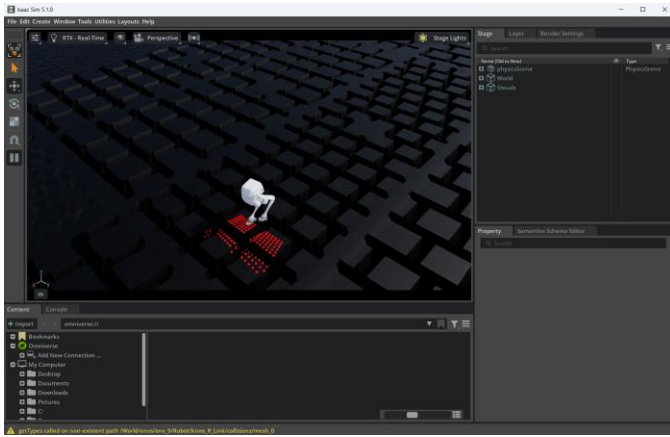


Fig.3 Simulation of bipedal locomotion on steppingstones in Isaac Sim.

The red spheres represent the sampled terrain scan points (11x18 grid) processed by the depth-based perception module, enabling the robot to identify gaps and adjust its foot placement via the residual policy.

A video demonstration of the robot traversing stepping-stone configurations in simulation is available at:

<https://www.youtube.com/watch?v=zYZK6G75fcM>.

B. Experiment

In the experiment, we deploy the policies already trained in simulation, real machines to the robot, observe whether they can operate as expected, and debug them.

1.Stage1 - blind walking gait on flat ground

Test content:

In this stage, after we deploy the flat-ground blind walking policy to the robot, Put it on level ground. To test which parameters affect the sim2real success rate, the requirements are large action scale and large torque output, while avoiding severe joint vibration.

Problems encountered:

We found that simulation accuracy, IMU angular velocity, joint angular velocity, and body pose will greatly affect the deployment performance of real aircraft.

IMU and joint angular velocity are very noisy, and the original data is put into the neural network in the real machine, and the output oscillation is huge. Filtering doesn't work either, there's a significant phase delay. The body pose has more influence, the pitch changes from 0 to 5 degrees, and the output action basically changes from 0 to the maximum value.

Solution:

1. Improve simulation accuracy
2. scale down the IMU angular velocity and joint angular velocity in obs so that reinforcement learning does not pay too much attention to angular velocity
3. Turn off the noise of the body position in the simulation. The noise in the body pose data given by the company is relatively small, so there is no need to add noise in the simulation.

2.Stage2 - traverse stepping-stone terrain

Test content:

In this stage, we enabled the depth camera and tested the actual effect of the policy for robots to perform the steps of the plum blossom pattern. In the test, we choose 190mm * 190mm * 390mm size hollow cement brick instead of plum pile. For each plum pile, we used four hollow cement bricks to form a cuboid platform of 780mm*380mm*190mm. Each plum-pile platform was placed in parallel with an interval of approximately 319.47mm. After placing the robot on the initial platform, the test starts.

Problems encountered:

We found that the robot occasionally stepped empty when walking the plum pile, and we speculated that the black hole on the side of the hollow brick affected the judgment of the depth camera of the robot.

Solution:

We used foam boards to cover up the holes in the cement brick. The robot was able to walk on the plum pile as expected.

Contribution:

Yiyu Liu:	Method
Deqing RUAN:	Simulation
Yimeng Zhang:	Conclusion
Gening Liu:	Experiment
Wuhao Cai:	Introduction

IV. CONCLUSION

This paper presented a reinforcement learning-based approach for bipedal locomotion on discontinuous stepping-stone terrain, where depth-based perception is directly integrated into the control policy. By formulating the locomotion problem as a Markov decision process and training the policy using Proximal Policy Optimization, the robot is able to learn stable and adaptive walking behaviors through interaction with the environment.

A key contribution of this work is the two-stage training framework, which significantly improves learning effectiveness and stability. In the first stage, the robot learns a reliable blind walking gait on flat ground, allowing the policy to acquire fundamental locomotion skills without the complexity of visual input. In the second stage, depth perception is introduced and a residual policy is trained to adapt the existing gait to discontinuous stepping-stone terrain. This staged strategy enables the robot to reuse previously learned walking behaviors and focus learning capacity on terrain adaptation, rather than relearning basic locomotion from scratch.

Simulation and real-world experiments demonstrate that the proposed reinforcement learning approach is effective for both flat-ground walking and stepping-stone traversal. The results show that policies trained with the two-stage framework can maintain stable walking while successfully selecting safe footholds in visually complex environments. Compared to directly training on challenging terrain, the staged training process leads to more robust performance and better sim-to-real transfer.

In summary, this work shows that deep reinforcement learning, when combined with a carefully designed two-stage training strategy, provides a practical and effective solution for bipedal locomotion on discontinuous terrain. These findings suggest that staged learning and residual adaptation are promising directions for applying reinforcement learning to complex robotic locomotion tasks.